

UNIVERSITA' TELEMATICA “e-Campus”

Facoltà di Ingegneria Informatica e dell'Automazione

Corso di Laurea in
Sistemi di Elaborazione e Controllo

**Progettazione e sviluppo di una piattaforma IoT
per il monitoraggio e l'analisi dei consumi
energetici negli edifici pubblici**

Relatore:

Prof. Kuznetsov Oleksandr

Tesi di Laurea di:

Vito Gagliano

Matricola n° 1857837

Anno Accademico 2025/2026

AUTORIZZAZIONE ALLA CONSULTAZIONE DELLA TESI DI LAUREA

Il/la sottoscritto Vito Gagliano

N° di matricola 1857837 nato a Catania il 09/11/1981

autore della tesi dal titolo: Progettazione e sviluppo di una piattaforma IoT per il monitoraggio e l'analisi dei consumi energetici negli edifici pubblici

AUTORIZZA

la consultazione della tesi stessa, fatto divieto di riprodurre, parzialmente o integralmente, il contenuto.

Dichiara inoltre di:

AUTORIZZARE

per quanto necessita l'università telematica e-Campus, ai sensi della legge n. 196/2003, al trattamento, comunicazione, diffusione e pubblicazione in Italia e all'estero dei propri dati personali per le finalità ed entro i limiti illustrati dalla legge.

Data_____

Firma



Ringraziamenti

Giunto al termine di questo percorso, sento il bisogno profondo di rivolgere lo sguardo a chi ha reso possibile ogni singolo passo.

Al mio relatore, il Professor Oleksandr Kuznetsov, va il mio più sentito ringraziamento per la costante disponibilità, la rapidità e la straordinaria precisione con cui mi ha seguito. Lo ringrazio con stima per avermi indirizzato verso un tema a me molto caro dal punto di vista professionale e per la fiducia che continua a dimostrarmi, incoraggiandomi verso la stesura di una futura pubblicazione scientifica.

Un grazie sincero va al mio amico e collega, Dott. Diego Sinitò, per il supporto costante, i preziosi consigli e il confronto continuo che mi hanno guidato nelle scelte progettuali di questa tesi.

Infine, il ringraziamento più importante è per mia moglie Arianna. Se oggi taglio questo traguardo, il merito è in gran parte tuo. Hai creduto nelle mie capacità sin dal primo giorno. Esame dopo esame, hai condiviso ogni momento di questo cammino con una pazienza infinita, sostenendomi e facendomi forza nei periodi più duri e carichi di impegni. Questa vittoria è anche frutto tuo. Grazie di esserci stata, sempre.

Indice

Introduzione	6
Capitolo 1 – Stato dell'arte e tecnologie di riferimento	10
1.1 IoT e Smart Building: architetture e paradigmi	
1.2 Protocolli di comunicazione per l'IoT (MQTT, CoAP, HTTP)	
1.3 Piattaforme IoT per il building management	
1.4 L'IoT nella Pubblica Amministrazione: normative e direttive europee sull'efficienza	
1.5 Tecniche di Anomaly Detection in ambito IoT: rassegna della letteratura	
1.6 Analisi comparativa degli algoritmi (Isolation Forest, One-Class SVM, Z-score)	
1.7 Dimensione dei modelli e deployment su dispositivi a risorse limitate (Edge Computing)	
Capitolo 2 – Progettazione dell'architettura del sistema	18
2.1 Requisiti di sistema e architettura generale	
2.2 Topologia della rete di sensori	
2.3 Scelta e giustificazione dello stack tecnologico	
2.4 Modello dei dati e flussi di comunicazione	
Capitolo 3 – Implementazione della Piattaforma AcquaSmart	25
3.1 Configurazione del broker MQTT e acquisizione dati	
3.2 Sviluppo del backend e logica di smistamento dati	
3.3 Realizzazione della dashboard web (Applicazione "tesi-acquasmart")	

3.4 Dataset utilizzato: dati reali e generazione tramite simulatore

3.5 Snippet di codice e configurazioni critiche

Capitolo 4 – Analisi dei dati e rilevamento anomalie 39

4.1 Selezione e configurazione dell'algoritmo Isolation Forest

4.2 Definizione del Ground Truth e metriche di valutazione (Precision, Recall, F1-Score)

4.3 Svolgimento degli esperimenti e valutazione delle performance (latenza, scalabilità)

4.4 Discussione dei risultati e analisi delle anomalie rilevate

Capitolo 5 – Conclusioni e sviluppi futuri 49

5.1 Valutazione critica dei risultati e raggiungimento degli obiettivi

5.2 Limiti del sistema proposto

5.3 Sviluppi futuri e integrazione con Building Management System (BMS)

5.4 Considerazioni finali

Bibliografia 53

Introduzione.

La progressione evolutiva delle tecnologie digitali ha determinato un sostanziale cambiamento nei processi produttivi, organizzativi e sociali, favorendo la nascita di nuovi modelli che si basano sull'automazione e condivisione intelligente. In tale contesto *l'Internet of Things (IoT)*[1] costituisce l'infrastruttura di base per la transizione digitale di molti ambienti fisici. Dotando di sensori e dispositivi intelligenti gli oggetti di uso quotidiano, i macchinari industriali, ecc., tramite l'utilizzo della IoT possiamo raccogliere e gestire un'enorme quantità di dati in tempo reale, creando così nuovi modelli gestionali.

Grazie allo *Smart Building Management*[2] è possibile ottimizzare in modo autonomo le prestazioni energetiche di un edificio civile o industriale. Se nel settore privato questa evoluzione è ormai consolidata, lo stesso non si può dire per la Pubblica Amministrazione (PA) Italiana, dove la transizione ancora è frammentata o del tutto assente. Eppure, esistono direttive europee sull'efficienza energetica e strumenti programmatici come il *PNRR* [3] e il Codice dell'Amministrazione Digitale, che impongono le PA a un radicale e repentino cambiamento in tal senso. All'interno di questo scenario orientato all'ottimizzazione dei consumi energetici globali degli edifici pubblici, il monitoraggio e la gestione delle risorse idriche si inseriscono come un caso applicativo specifico e di prioritaria importanza. In territori caratterizzati da frequenti e prolungate carenze di acqua potabile, la riduzione delle dispersioni idriche è fondamentale non solo per mitigare una criticità oggettiva, ma anche danni economici rilevanti. Implementare un sistema di monitoraggio tramite l'IoT negli edifici pubblici significherebbe superare la manutenzione reattiva e adottare un modello predittivo guidato dai dati (*data-driven*)[4]. L'ideazione e progettazione del sistema presentato in questo elaborato non rappresenta solo un esercizio accademico, bensì nascono da un'esigenza operativa e concreta che è

stato riscontato e affrontato sul campo, in risposta alle criticità gestionali rilevate dalla direzione Patrimonio e Politiche Scolastiche della *Città Metropolitana di Catania* [5]. L'Ente preso in esame ha in gestione la manutenzione (ordinaria e straordinaria) di 63 edifici scolastici secondari di secondo grado. A causa della sua vasta estensione, il territorio catanese è ripartito in quattro macro-aree (Metropolitana, Ionica, Pedemontana e Calatina). Su tale superficie, la gestione tecnica del patrimonio scolastico è resa particolarmente complessa da una forte frammentazione logistica: non è raro, infatti, che un singolo istituto comprenda una sede principale situata nel capoluogo e diverse succursali dislocate in comuni periferici o in aree geografiche completamente differenti. Allo stato attuale, il monitoraggio dei consumi idrici pecca di una grave inefficienza amministrativa. Infatti, i contatori non sono intestati a un'unica anagrafe centralizzata, bensì ai singoli istituti scolastici. Le fatturazioni, spesso basate su stime di consumo e soggetti a conguagli o oneri di mora, rendono difficile, se non quasi impossibile, la gestione delle risorse. In fase di analisi, abbiamo valutato l'unificazione contrattuale di tutti i contatori, ma questa soluzione richiedeva un iter burocratico troppo lungo per un'esigenza così urgente. Da qui è nata la necessità di una soluzione rapida che aggirasse tutti i vincoli. La totale assenza di sistemi di telelettura, che impedisce di rilevare tempestivamente micro-perdite o rotture accidentali (spesso scoperte solo a fatto avvenuto), mi ha portato a creare il sistema *AcquaSmart*. Grazie all'installazione di flussimetri intelligenti agli ingressi delle reti idriche principali e secondarie, ho fatto in modo che tutti i dati convogliassero verso un'infrastruttura Cloud, potendo così sviluppare una dashboard operativa (disponibile all'indirizzo <https://tesi-acquasmart.onrender.com>). Così, per la prima volta, l'ente dispone di uno strumento di

controllo capillare e in tempo reale, capace di abbattere drasticamente i tempi di intervento.

Per rispondere a questa sfida, il presente lavoro di tesi si è concentrato sulla progettazione, sullo sviluppo e sulla verifica sperimentale di *AcquaSmart*: una piattaforma IoT chiavi in mano (end-to-end) per la raccolta e l'analisi in maniera automatizzata dei consumi negli edifici pubblici. Al fine di distinguere chiaramente il perimetro dell'implementazione, è fondamentale precisare che la piattaforma software (architettura cloud, database, algoritmi di Machine Learning e dashboard utente) è stata interamente sviluppata ed eseguita verifica sperimentale. Mentre la rete fisica dei sensori (flussometri intelligenti) sono oggetto di simulazione software, al fine di generare dei flussi di dati verosimili alla realtà per testare l'infrastruttura. L'installazione fisica dei dispositivi fisici sul campo sarà oggetto di sviluppo futuro. Il valore aggiunto non è da ricercarsi nella semplice digitalizzazione del dato, ma nell'utilizzo di algoritmi di *Machine Learning* [6] come supporto decisionale, i quali permettono di analizzare i flussi in tempo reale per individuare tempestivamente anomalie, micro-perdite o utilizzi imprevisti all'interno della rete idrica.

La mia ricerca si è articolata in diverse fasi operative. Inizialmente, ho condotto un'analisi critica sullo stato dell'arte, esaminando le piattaforme già esistenti per il *building management* [7] e ponendo particolare attenzione ai protocolli di messaggistica leggeri e alle tecniche di *anomaly detection* [8] sui dati storici. Successivamente, sono passato alla progettazione dell'architettura di sistema, definendo un flusso dati sicuro e scalabile che parta dai sensori Edge, passi per i *broker MQTT* [9] e, infine, approdi al database di backend.

L'attività è proseguita con lo sviluppo e il deployment del prototipo, mettendo molta attenzione all'interfaccia utente, affinché risultasse di facile utilizzo per gli operatori dell'ente. Un modulo centrale del progetto è costituito nell'implementazione dell'algoritmo di *Isolation Forest*[10], scelto per la sua velocità di calcolo su flussi dati non supervisionati. Infine, ho completato il lavoro con una validazione sperimentale quantitativa, misurando le latenze dell'architettura e verificando l'accuratezza del modello nel distinguere pattern di consumo anomali, minimizzando così i falsi positivi.

L'elaborato si sviluppa in sei capitoli che guideranno il lettore dalla definizione teorica del problema fino ai risultati sperimentali.

Nel **Capitolo 1** si delinea lo stato dell'arte, analizzando il paradigma IoT e il quadro normativo di riferimento, e ponendo le basi teoriche sulle tecniche di rilevamento anomalie attraverso la letteratura scientifica.

Il **Capitolo 2** è dedicato alla progettazione concettuale. Vengono definiti i requisiti di sistema, giustifico la scelta dello stack tecnologico, (tra cui il broker MQTT e il framework di backend) e illustro il modello relazionale dei dati.

Il **Capitolo 3** descrive la fase di implementazione pratica di *AcquaSmart*, illustrando le configurazioni server, i meccanismi di acquisizione e l'architettura della dashboard operativa.

Il **Capitolo 4** descrive le procedure di addestramento del modello, viene definito il *Ground Truth* e presentato un'analisi critica delle performance basata su metriche standard come Precision, Recall e F1-Score.

Il **Capitolo 5** traccia le conclusioni, valutando il raggiungimento degli obiettivi e definendo una roadmap per futuri sviluppi, in particolare per l'integrazione della piattaforma nei sistemi BMS già in uso presso l'ente.

Capitolo 1 – Stato dell'arte e tecnologie di riferimento

1.1 IoT e Smart Building: architetture e paradigmi

In un passato recente, i sistemi di automazione degli edifici si basavano su logiche molto rigide. I tradizionali *Building Management System* (BMS)[7] venivano concepiti come compartimenti stagni che lavoravano esclusivamente su reti cablate, spesso fisicamente inaccessibili, e protocolli proprietari che rendevano quasi impossibile far dialogare sistemi diversi tra loro. L'avvento dell'Internet of Things (IoT) ha portato flessibilità dove prima c'era chiusura. In contesti operativi complessi, dove si gestiscono decine di edifici storici o frammentati sul territorio, questo cambiamento ha portato enormi vantaggi.

Le nuove strategie IoT hanno superato i limiti fisici del passato, basandosi su reti distribuite di sensori e attuatori (i cosiddetti *Cyber-Physical Systems*[11]), capaci di comunicare utilizzando standard aperti. In un'infrastruttura reale e funzionante, l'architettura di uno Smart Building viene solitamente suddivisa in tre livelli logici ben distinti:

- **Perception Layer (Livello Fisico):** È il punto di contatto con l'ambiente reale. È costituito dall'hardware installato fisicamente sul campo, come i flussimetri intelligenti posizionati sui tubi e i microcontrollori che hanno il compito di intercettare il dato grezzo direttamente dalla rete idrica.
- **Network Layer (Livello di Rete):** Responsabile della trasmissione sicura e affidabile dei dati, utilizzando connettività wireless o filare (cablata)

- **Application Layer (Livello Applicativo):** È il vero "cervello" del sistema, ospitato su un server *on-premise* o in Cloud. Qui i dati vengono aggregati, storicizzati e analizzati per fornire strumenti concreti a chi gestisce la manutenzione, come dashboard di monitoraggio visivo e sistemi di allerta (alert) automatici in caso di perdite.

Il grande vantaggio operativo di questo modello a strati è la sua naturale scalabilità. Permette, infatti, di partire con un progetto pilota per poi integrare progressivamente plessi scolastici. Basterà quindi aggiungere nuovi nodi alla rete periferica, senza mai dover stravolgere o riprogettare l'infrastruttura centrale di base.

1.2 Protocolli di comunicazione per l'IoT (MQTT, CoAP, HTTP)

La scelta del protocollo di messaggistica è cruciale soprattutto in determinati scenari. I protocolli del web a volte sono inadeguati e per tale motivo si devono ricercare soluzioni ottimali

Protocollo	Trasporto	Modello di Comunicazione	Overhead	Caso d'uso ideale
HTTP(S)	TCP	Request/Response	Alto	Scambio documentale, API web standard.
CoAP	UDP	Request/Response	Basso	Reti con alta latenza e alta perdita di pacchetti.
MQTT	TCP	Publish/Subscribe	Molto Basso	Telemetria continua, sensori a bassa potenza,

Protocollo	Trasporto	Modello di Comunicazione	Overhead	Caso d'uso ideale
				push di dati in tempo reale.

Tabella 1 Analisi comparativa dei protocolli di rete (HTTP, CoAP, MQTT) per le architetture IoT.

Tra le opzioni riportate in tabella 1, *MQTT* (*Message Queuing Telemetry Transport*[9]) si è ormai imposto come lo standard indiscusso per le applicazioni IoT, sia in ambito industriale che civile. Il suo vero punto di forza risiede nell'architettura. La struttura è centralizzata sul *Broker*, che ha il compito di smistare i messaggi. Questo meccanismo separa nettamente chi produce l'informazione (il *Publisher*, nel nostro caso il sensore sul tubo) da chi la deve leggere o elaborare (il *Subscriber*, ovvero il server o la dashboard). In situazioni critiche delle strutture idriche dove la copertura di rete è scarsa, utilizzare sensori MQTT ci viene incontro perché trasmette “pacchetti” dati leggerissimi e soprattutto protegge i dati dai cali di connessione evitando che il sistema vada in blocco in caso di disconnessione e trasmettendo i dati non appena il dispositivo torna online.

1.3 Piattaforme IoT per il building management

Analizzando le attuali opzioni disponibili sul mercato, si trova un'ampia scelta di sistemi per il *building management*. Soluzioni commerciali offerte da AWS IoT o Microsoft Azure IoT, o architetture open-source come ThingsBoard o Home Assistant. Naturalmente questi sistemi vantano un'infrastruttura solida con potenti strumenti integrati per l'analisi dei dati. Eppure il loro modello può risultare inadatto ad alcune Amministrazioni Pubbliche sia per i costi sostenuti delle licenze annuali, sia per gli strumenti da essi proposti, il più delle volte standardizzati su parametri già prefissati e

magari non rispettando le reali necessità dell'ente. È per superare questi ostacoli che nasce il progetto *custom* adottato in questo lavoro di tesi. Impiegando framework di sviluppo moderni e allo stesso tempo “leggeri”, è stato possibile costruire da zero una piattaforma sagomata sulle reali esigenze di monitoraggio idrico degli edifici scolastici. Uno strumento scalabile capace di essere implementato o modificato secondo le esigenze future, totalmente libero da licenze proprietarie ed esente da vincoli contrattuali di assistenza annuale.

1.4 L'IoT nella Pubblica Amministrazione: normative e direttive europee sull'efficienza

Rendere gli edifici pubblici più efficienti e digitalizzati, non rappresenta una semplice opzione di ammodernamento tecnologico, ma risponde a un preciso obbligo normativo. Concentrandoci su uno scenario europeo, è proprio la Direttiva sulle Prestazioni Energetiche degli Edifici (*EPDB- Energy Performance of Buildings Directive*[12]) ad accelerare l'impiego di sistemi intelligenti capaci di misurare e governare i consumi in modo minuzioso. Anche sul fronte nazionale questa direttiva Europea ha trovato un'applicazione attraverso il *Piano Nazionale di Ripresa e Resilienza (PNRR)*. Attraverso le Missioni 1 e 2 del medesimo Piano, sono stati destinati considerevoli fondi economici alla transizione digitale delle P.A. e alla riqualificazione del patrimonio immobiliare. A questo si affianca il *Codice dell'Amministrazione Digitale (CAD)*[13], che sancisce che le scelte devono essere guidate dai fatti (*data-driven decision making*). Questo significa che le istituzioni sono chiamate a basare i propri processi amministrativi e manutentivi su dati oggettivi e misurabili e non approssimativi. Considerato ciò, affidarsi alla lettura stimata dei contatori idrici è una pratica che deve essere superata definitivamente.

Pertanto diventa una tappa obbligatoria l'utilizzo di sistemi di telemetria e monitoraggio automatizzato.

1.5 Tecniche di Anomaly Detection in ambito IoT: rassegna della letteratura

L'acquisizione dei dati è il punto di partenza. La vera sfida consiste nel trasformarli in valore operativo. Quando parliamo di efficienza idrica all'interno degli istituti scolastici, lo scopo fondamentale è trovare tempestivamente guasti, micro-perdite o utilizzi imprevisti (flussi d'acqua durante la notte o giorni di chiusura degli istituti). Dal punto di vista tecnico questi tipi di problemi rientrano nel campo dell'*Anomaly Detection* (*rilevamento delle anomalie*) applicato alle serie storiche. Analizzando gli studi di settore, per intercettare queste deviazioni si possono individuare tre macro-categorie:

1. Sistemi basati su regole (Rule-based): Un approccio tradizionale, che prevede l'impostazione di soglie massime prefissate o finestra temporali di allarme. Immediati e facili da implementare ma a volte troppo rigidi. Tendono a generare un alto numero di falsi positivi, soprattutto quando per questioni interne all'istituto scolastico variano i consumi abituali, rendendo il sistema di allerta poco credibile per chi deve gestirlo.

2. Modelli Supervisionati: Sono degli algoritmi di Machine Learning che vengono addestrati partendo da un set di dati già etichettati preventivamente come un consumo fisiologico o un'anomalia. Nella realtà operativa di un ente pubblico, questa soluzione è quasi sempre impercorribile, visto che è improbabile avere un database storico dettagliato su cui poter addestrare il sistema in modo efficace.

3. Modelli Non Supervisionati: In questo caso, l'algoritmo è in grado di studiare autonomamente lo storico dei dati. Deduce quindi quale sia il comportamento abituale

dell'impianto segnalando solo le misure che si discostano in modo netto dalla normalità. Proprio per la flessibilità e adattamento in assenza di dati storici, questa famiglia di algoritmi rappresenta la soluzione più concreta e idonea per i flussi di telemetria IoT

1.6 Analisi comparativa degli algoritmi (Isolation Forest, One-Class SVM, Z-score)

Per individuare il motore più adatto all'architettura di AcquaSmart, ho condotto un'analisi comparativa dei principali algoritmi non supervisionati.

Z-Score (approccio Statistico)

Questo algoritmo quantifica di quante deviazioni standard una determinata misurazione si discosta dal valore medio della distribuzione. Da un punto di vista matematico, possiamo definire:

$$z = (x - \mu) / \sigma$$

Il suo principale vantaggio è senz'altro l'estrema leggerezza di calcolo. Tuttavia, questo metodo assume che i dati seguano una distribuzione normale (*Gaussiana*). I consumi idrici all'interno di un istituto scolastico, presentano invece dinamiche non lineari. Infatti si presentano spesso picchi improvvisi o cali drastici dei consumi in base a giornate, ore o attività scolastiche o extra scolastiche. Pertanto questo algoritmo è fortemente inadeguato per il nostro contesto.

One-Class SVM (Support Vector Machine)

Questo algoritmo cerca di disegnare un confine sferico attorno alle misure "normali", proiettandole in uno spazio a più dimensioni. Sebbene garantisca un'elevata precisione nel riconoscimento dei modelli complessi, presenta un grosso limite architetturale nel consumo delle risorse di calcolo. Quindi è risaputo che al crescere del volume dei dati da

elaborare i tempi di calcolo aumentano drasticamente. Una simile latenza non è accettabile e compatibile con un sistema che lavora in tempo reale.

Isolation Forest

Questo algoritmo ribalta la logica dei metodi di rilevamento tradizionali. Anziché analizzare matematicamente il comportamento “normale”, mira a isolare in modo diretto le anomalie. Il modello sfrutta un insieme di alberi decisionali casuali, per ricavarne previsioni molto precise. Quindi si deduce che i dati anomali sono rari e presentano valori distanti dal *dataset*. Questa dinamica si traduce in una straordinaria efficienza prestazionale, garantendo una complessità temporale di tipo logaritmico:

$$O(n \log n)$$

Per la propria rapidità di esecuzione, la capacità di adattamento alla variazioni periodiche dei consumi idrici senza richiedere ulteriori addestramenti dell'algoritmo, si è scelto di utilizzare Isolation Forest come motore di rilevamento di AcquaSmart.

1.7 Dimensione dei modelli e deployment su dispositivi a risorse limitate (Edge Computing)

Un'ulteriore considerazione progettuale di rilievo è stata rivolta alla distribuzione del carico computazionale. Anche se le piattaforme Cloud mettono a disposizione una capacità di calcolo teoricamente inesauribile, trasmettere ogni singola misurazione grezza dai vari sensori IoT al server centrale per applicare il modello di calcolo, rischia di saturare la banda di rete, introducendo della latenza del tutto evitabile.

Il modello dell'*Edge Computing*, risolve questa criticità spostando la fase di elaborazione o di esecuzione dei modelli pre-addestrati direttamente a ridosso delle sorgenti dei dati (gateway o IoT locali). È proprio in questo scenario che algoritmi come

Isolation Forest si rivelano veramente vantaggiosi: il loro ridotto impatto sulla memoria (memory footprint) ne facilita l'implementazione anche su dispositivi con risorse hardware limitate.

Capitolo 2 – Progettazione dell’architettura del sistema

Progettare l’infrastruttura tecnologica per la piattaforma “AcquaSmart” ha richiesto prima di tutto un’analisi accurata delle necessità tecniche e dei limiti fisici degli edifici scolastici coinvolti. In questo capitolo si descriveranno le scelte architetture alla base del progetto, i motivi che hanno spinto verso determinate tecnologie e il modello dei dati che gestisce lo scambio di informazioni tra i diversi moduli del sistema.

2.1 Requisiti di sistema e architettura generale

Sviluppare una piattaforma IoT per la gestione degli edifici pubblici (building management) significa dover rispettare standard molto rigidi in termini di affidabilità, scalabilità e sicurezza.

Nella fase iniziale di raccolta dei requisiti, sono delineati i seguenti obiettivi pratici e funzionali:

- **Acquisizione in tempo reale (Real-time data ingestion)[14]:** il sistema ha il compito di monitorare continuamente lo stato delle reti idriche. Per farlo deve ricevere ed elaborare grandi quantità di dati da remoto con latenze minime.
- **Rilevamento intelligente delle anomalie (Anomaly Detection):** oltre a salvare i dati, l’applicativo deve valutare ogni singola misurazione, per identificare variazioni significative rispetto al comportamento nominale (perdite occulte, guasti o consumi fuori orario), utilizzando sia logiche a soglia fissa, sia modelli di Machine Learning.

- **Scalabilità e disaccoppiamento:** l'architettura è pensata per espandersi facilmente. Deve supportare sia l'inserimento di nuovi istituti scolastici, sia l'installazione quindi monitoraggio di centinaia di nuovi sensori, senza richiedere interventi strutturali sul nucleo del sistema.

- **Accessibilità e sicurezza:** le informazioni e i dati, devono essere consultabili dagli operatori tecnici e anche dai dirigenti della Pubblica Amministrazione attraverso una dashboard intuitiva e sicura, accessibile via web senza alcun vincolo di installazioni software o di client dedicati.

Per far sì che tutti questi requisiti fossero soddisfatti, è stata adottata un'architettura a livelli (multi-tier). Soluzioni simili sono già utilizzate e consolidate negli ecosistemi IoT più moderni. Il sistema "AcquaSmart" è strutturato in 3 macro-livelli connessi tra loro:

1. **Livello di Campo (Edge):** comprende l'hardware di rilevamento fisico, come flussimetri intelligenti e sonde di monitoraggio (pressione, torbidità, conducibilità) installati presso i vari snodi idrici dei plessi scolastici.

2. **Livello di Trasporto (Middleware):** costituito da un broker per la gestione personalizzata della messaggistica, che funge da collegamento tra le sonde sul campo e l'infrastruttura server.

3. **Livello Applicativo (Backend e Frontend):** rappresenta il motore centrale della piattaforma, preposto all'aggregazione dei dati, all'elaborazione predittiva, all'archiviazione e della visualizzazione tramite API RESTful.

Questa scelta architetturale favorisce l'affidabilità del sistema: infatti, eventuali malfunzionamenti o di una sonda periferica o manutenzioni sul database non compromettono la stabilità e l'integrità dell'intera infrastruttura.

2.2 Topologia della rete di sensori

Il monitoraggio dei consumi idrici nelle scuole superiori impone l'adozione di una topologia di rete capace di mappare il più fedelmente possibile l'infrastruttura idrica. Per tale motivo, la rete di sensori è stata progettata secondo un modello ibrido a stella-albero.

Ciascun plesso scolastico si configura come un nodo primario di aggregazione logica. In corrispondenza dell'allaccio principale all'acquedotto Sidra S.P.A. (società idrica Catanese), è prevista l'installazione di un sensore principale (chiamato Contatore Principale o Master-Sensor), che ha il compito di misurare il consumo idrico complessivo dell'istituto. A valle di questo misuratore, la rete si dirama verso i sotto-sensori (chiamati Sensori secondari o Sub-Sensors), distribuiti secondo una logica ben precisa nei punti di erogazione a maggior rischio o consumo (esempio: servizi igienici di piano, spogliatoi palestre, laboratori e nelle vasche di raccolta per il sistema antincendio).

Tale suddivisione permette di ottenere informazioni estremamente dettagliate e, soprattutto in presenza di anomalie, consente di isolare fisicamente l'area interessata dal guasto tramite un raffronto incrociato con i sotto-sensori, riducendo drasticamente i tempi di intervento per la manutenzione.

Oltre ai misuratori di portata volumetrica (Acqua), sono state previste sonde per il controllo della qualità (torbidità e durezza) e della stabilità (pressione) dell'impianto. L'infrastruttura si configura pertanto come uno strumento multifunzionale: le misure di pressione monitorano continuamente l'efficienza degli impianti di spinta (autoclavi), mentre i valori ottenuti dai sensori di torbidità e conducibilità controllano costantemente lo stato igienico-sanitario dell'acqua.

2.3 Scelta e giustificazione dello stack tecnologico

Le scelte architetturali alla base della piattaforma sono state dettate dalla necessità di realizzare un sistema robusto, performante e basato su tecnologia open-source, liberando l'Amministrazione da vincoli commerciali e licenze proprietarie.

Per l'instradamento e la gestione dei flussi telemetrici, si è optato per *Mosquitto* [15] come broker MQTT (in ascolto sulla porta standard 1883). Questa scelta nasce dal ridotto impatto sulle risorse di memoria e dalla capacità di mantenere stabili migliaia di connessioni concorrenti. Il broker assolve esclusivamente alla funzione di instradare i messaggi sui relativi topic (canali telematici), demandando ai livelli superiori la logica applicativa e compensando le frequenti instabilità di connessione che si manifestano nei locali tecnici delle scuole.

Il nucleo logico (Backend) è stato sviluppato interamente in *Python*[16] sfruttando il framework *FastAPI*[17]. La decisione di preferire Python ad altri linguaggi tradizionali utilizzati solitamente nel web enterprise (Java o C) è motivata da due esigenze tecnologiche. L'adozione di FastAPI risponde a due necessità primarie: la gestione efficiente e nativa delle operazioni di I/O asincrone, supportata dalla sua architettura ASGI, e la totale interoperabilità con le librerie dedicate al Machine Learning. Il rilevamento delle anomalie tramite l'algoritmo *Isolation Forest* [10] è basato su librerie scientifiche quali "scikit-learn" e "pandas"; grazie a Python, l'elaborazione dell'intelligenza artificiale avviene in modo fluido e integrato all'interno del sistema principale. Questo sistema ha evitato i rallentamenti causati dai continui scambi dati tra i vari servizi.

Per il salvataggio dei dati è stato seguito un approccio relazionale ibrido, implementato in modo fluido grazie a *SQLAlchemy*[18]. L'architettura dati è stata progettata per operare in maniera fluida e veloce su un database come *SQLite*[19], ideale per fasi di test, prototipazione e per una messa in funzione del software destinata a pochi utenti. Inoltre, l'uso dell'*ORM (object-relational Mapping)* [20] , offre una grande flessibilità, infatti permetterà di passare a un database più potente *PostgreSQL* senza dover riscrivere il codice (zero-rewrite). Un vantaggio fondamentale per preparare il sistema a gestire grandi quantità di dati per una crescita del progetto negli anni a venire.

Infine, la parte visiva dell'applicazione (frontend) è una dashboard web sviluppata utilizzando tecnologie standard come HTML, CSS e JavaScript (Vanilla JS). Questa soluzione garantisce un'interfaccia leggera, reattiva e ottimizzata per tutti i browser e i dispositivi mobili, servita direttamente dal server senza la necessità di ricorrere a complessi framework aggiuntivi.

2.4 Modello dei dati e flussi di comunicazione

L'efficacia di un'applicazione guidata dai dati dipende fortemente dalla stabilità del suo modello relazionale e dalla sicurezza dell'infrastruttura. Seguendo questa logica, il database di "*AcquaSmart*" è stato strutturato per massimizzare le prestazioni: da un lato velocizzando le interrogazioni sui dati anagrafici e temporali, dall'altro ottimizzando l'estrazione delle serie storiche relative alle letture.

Il modello dei dati si articola su quattro componenti fondamentali:

- **Scuole:** gestisce l'anagrafica degli istituti, memorizzando indirizzi, codice meccanografico e numero degli alunni.

- **Sensori:** l'entità che ha una relazione gerarchica (1 a N) con le scuole dove N rappresenta le scuole. Essa associa ogni dispositivo hardware al proprio identificativo di rete (topic MQTT) e alla tipologia di misurazione (Acqua, Pressione, ecc.), stabilendo tramite flag logici che ruolo topologico del sensore (principale o secondario).

- **Letture:** rappresenta la tabella più operativa e sollecitata del sistema. Archivia ogni singola rilevazione, associando al dato quantitativo il momento esatto della misurazione e l'output predittivo dell'intelligenza artificiale (eventuali anomalie e probabilità).

- **Utente e Dati Reali:** due sottosistemi di supporto cruciali per l'applicazione. Da un lato assicurano un accesso controllato e sicuro alla piattaforma tramite *JSON Web Token*[21], gestendo i vari ruoli utenti (amministratore o guest), dall'altro immagazzinano i consumi reali documentati dalle bollette (indispensabili per avere un termine di paragone per il sistema).

L'intero ciclo di vita del dato è stato concepito come un flusso continuo e asincrono, strutturato per prelevare la misurazione dal campo (flussimetro) e farla visualizzare dall'utente sulla dashboard, attraverso cinque passaggi fondamentali:

1. **Acquisizione:** i dispositivi hardware inviano al broker i flussi di dati in formato JSON. La gerarchia della rete idrica viene mappata sfruttando i topic di comunicazione e garantendo un indirizzamento pulito e scalabile.

2. **Elaborazione:** un servizio sempre attivo all'interno del server FastAPI intercetta questi messaggi; successivamente il backend si fa carico di spaccettare il dato grezzo, validarlo e collegarlo all'esatta identità del sensore che lo ha generato.

3. Inferenza: il dato normalizzato viene quindi immediatamente sottoposto al modello di Isolation Forest, che elabora i flussi idrici calcolando in tempo reale la probabilità che si tratti di un'anomalia. Per i dati non idrici (torbidità, pressione, conducibilità) vengono invece applicate delle rigide soglie normative.

4. Persistenza e Reazione: ogni singola lettura ricevuta insieme al proprio risultato predittivo viene storicizzata nel database. Se l'algoritmo conferma uno stato anomalo, il sistema non si limita a salvare l'informazione, ma attiva un allarme sulla stessa interfaccia di monitoraggio e invia tempestivamente delle mail ai tecnici manutentori.

5. Presentazione: i dati grezzi vengono aggregati su base oraria, giornaliera e mensile ed esposti alla Web-Dashboard in sicurezza tramite endpoint REST, con il compito di trasformare una grande quantità di dati complessi in visualizzazioni grafiche semplici e intuitive.

Capitolo 3 – Implementazione della Piattaforma

“Acquasmart”

L’implementazione del sistema “Acquasmart” ha tradotto in realtà operativa i concetti teorici e le scelte architetturali delineate nel capitolo precedente. In questa fase, le tecnologie selezionate sono state integrate per costruire un flusso di dati fluido e affidabile, dal singolo sensore posizionato sulla rete idrica fino all’interfaccia utente finale. Questo capitolo quindi illustra nel dettaglio le procedure di configurazione, lo sviluppo del backend e della dashboard, la strategia per la simulazione del dataset, arricchendo il contenuto con diagrammi illustrativi e porzioni di codice essenziale (*Listato3.X*).

3.1 Configurazione del broker MQTT e acquisizione dati

Il sistema di telemetria di “AcquaSmart” si fonda sull’utilizzo del protocollo MQTT[15]. Per garantire l’instradamento in tempo reale delle letture, si è scelto di utilizzare Mosquitto come broker. La sua natura leggera e la sua elevata scalabilità lo rendono ideale per gestire un flusso continuo di dati (*data stream*) provenienti da numerosi istituti scolastici.

Architettura di acquisizione: Di seguito viene presentato lo schema dell’architettura generale del sistema, che illustra il percorso del dato dal livello di campo al livello applicativo:

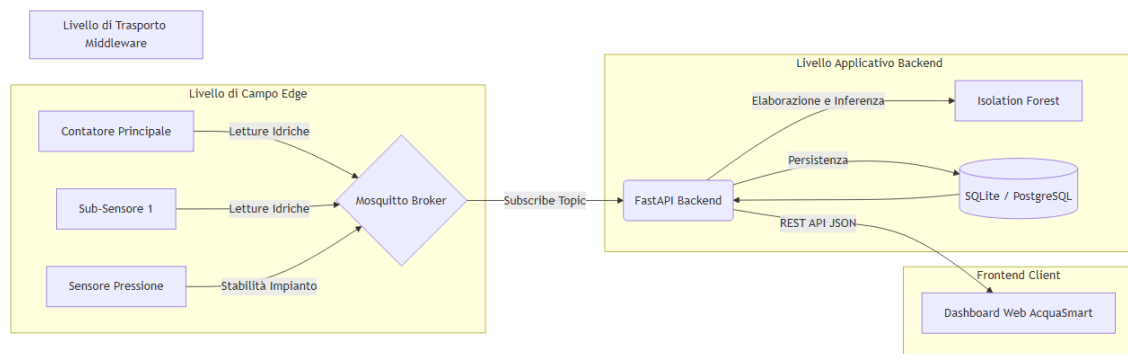


Figura 3.1: AcquaSmart: Architettura IoT Edge-to-Cloud per il monitoraggio idrico e il rilevamento predittivo di anomalie negli edifici scolastici.

Per rappresentare la topologia dei nodi sensore si è adottata una mappatura diretta all'interno dell'architettura dei topic MQTT. Ad esempio, la ricezione dei dati da una specifica scuola segue una classificazione gerarchica tipo: `'/catania/scuole/{id_scuola}/{id_sensore}'`, permettendo al backend di filtrare e indirizzare in modo efficiente le informazioni. L'acquisizione dei dati è garantita in modo asincrono, evitando blocchi (colli di bottiglia), nell'eventualità in cui si verifichino congestioni impreviste.

3.2 Sviluppo del backend e logica di smistamento dati

Il backend costituisce il motore logico dell'infrastruttura. Realizzato in Python avvalendosi di FastAPI, ad esso è demandato il coordinamento del traffico dati, la preparazione delle feature (attributi) per i modelli di Machine Learning e la persistenza affidabile e sicura dei record.

Una volta che il broker MQTT riceve il messaggio dal sensore, il backend (che è in continuo ascolto sul canale di comunicazione prestabilito) intercetta il pacchetto

(*payload*), che solitamente si trova in formato *JSON*. La logica di smistamento dati si articola nelle seguenti fasi:

1 Validazione ed Estrazione: il dato grezzo viene estratto e validato per evitare inconsistenze.

2 Arricchimento del dato (Data Enrichment): alla lettura del dato viene subito associato l'orario di acquisizione (*timestamp*) e l'identificativo esatto del sensore (bagno, vasca, ecc.) interrogando preventivamente il database.

3 Elaborazione e Inferenza: prima del salvataggio, il dato idrico viene sottoposto al modulo predittivo (Isolation Forest) per calcolarne il livello di anomalia (*anomaly score*).

4 Persistenza (Storage): l'entità completa, integrata dai risultati dell'algoritmo, viene salvata nel database relazionale.

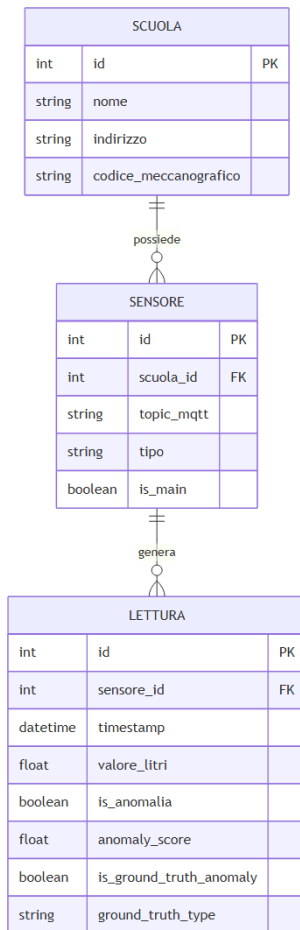


Figura 3.2 – Schema Mermaid per la definizione del Modello Entità-Relazione e dei vincoli di chiave.

3.3 Realizzazione della dashboard web (Applicazione “tesi-AcquaSmart”)

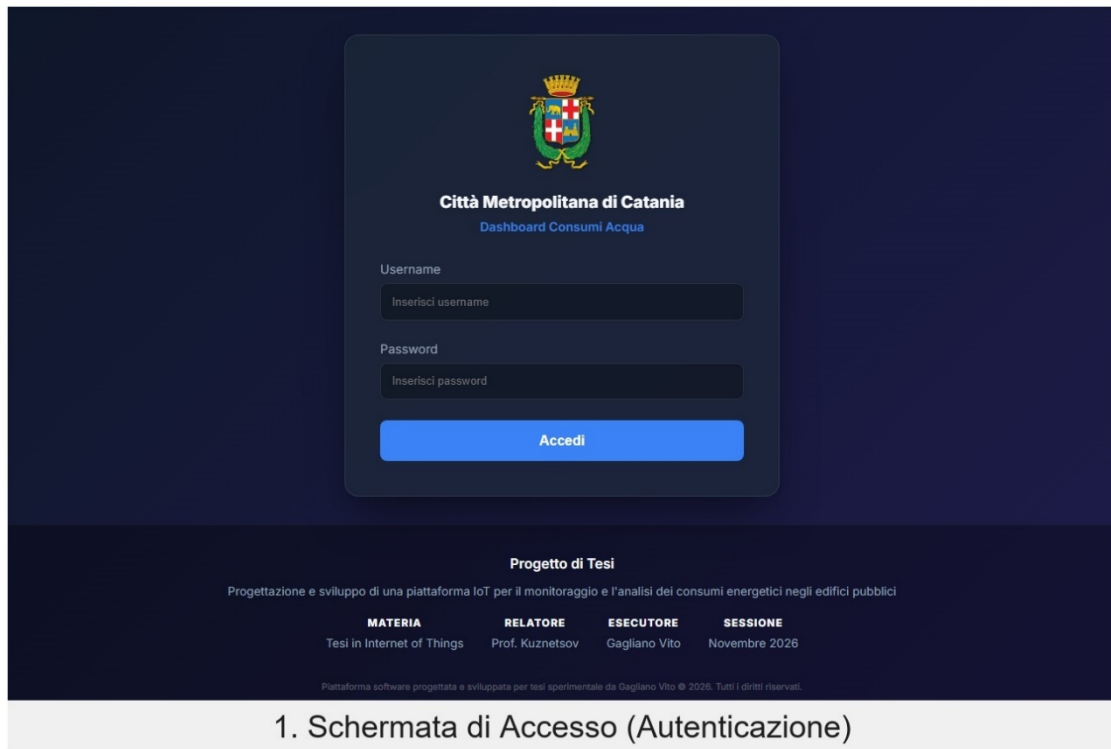
L’interfaccia utente rappresenta il mezzo principale mediante cui l’amministrazione pubblica può convertire l’astrazione dei flussi di dati in operazioni immediate. Il pannello di controllo web è stato progettato con un’estetica essenziale e strettamente focalizzata sulle informazioni (*data-driven*); la sua implementazione si avvale di tecnologie web native (*HTML, CSS, JavaScript Vanilla*) al fine di garantire un’elevata responsività e una totale interoperabilità su qualunque browser o terminale mobile.

L’applicazione comunica esclusivamente con il backend tramite API RESTful protette, implementando un solido sistema di sicurezza. L’accesso alla piattaforma è infatti vincolato a una schermata di autenticazione iniziale (Login) che richiede credenziali verificate, tutelando così i dati sensibili dell’Ente.

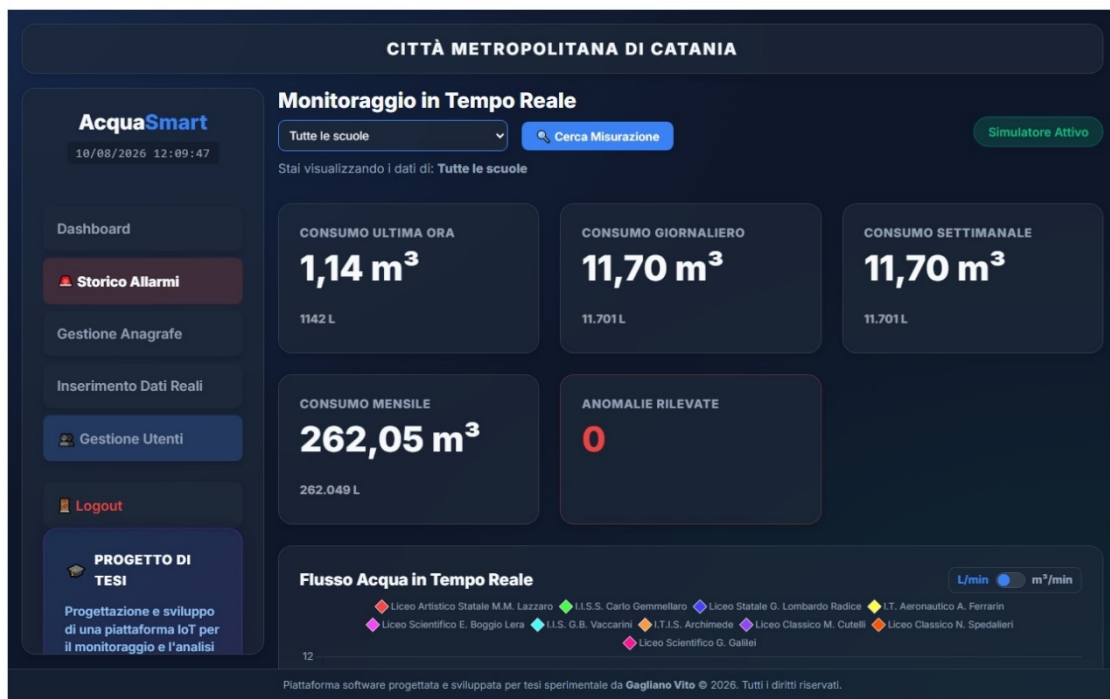
All'interno del portale, le autorizzazioni sono gestite in base ai ruoli: gli utenti dotati di privilegi di amministratore dispongono di un pannello di controllo avanzato tramite il quale è possibile gestire dinamicamente l'infrastruttura, apportare modifiche all'anagrafica, registrare nuove scuole, aggiungere ulteriori sensori IoT alla rete e creare o revocare utenze per gli operatori.

L'architettura si articola nei seguenti moduli principali:

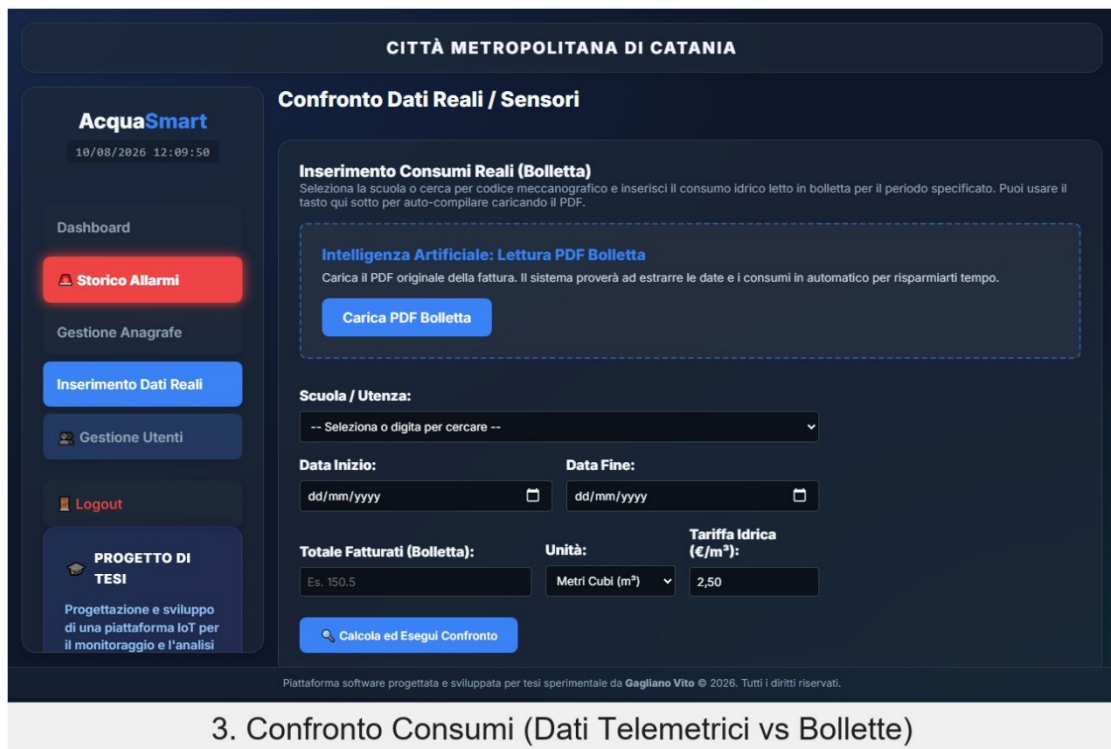
- **Vista Globale (Overview):** restituisce i consumi idrici consolidati e il monitoraggio in tempo reale dello stato operativo e di allerta dell'intero parco scolastico.
- **Dettaglio del plesso:** fornisce rappresentazioni grafiche dinamiche che tracciano l'andamento temporale (serie storiche) dei flussi per ogni nodo sensore, consentendo un'analisi di precisione.
- **Confronto Dati Reali (Bollette):** un'area dedicata all'inserimento dei dati reali di fatturazione, permettendo al sistema di incrociare le letture telemetriche stimate con i consumi effettivi documentati dalle bollette idriche fornendo un termine di paragone essenziale anche in termine economico.
- **Log degli allarmi (Alerting):** area dedicata agli eventi critici identificati dal modello predittivo, indispensabile per indirizzare i tecnici manutentori nell'isolamento fisico delle diramazioni d'impianto soggette a perdita o avarie.



1. Schermata di Accesso (Autenticazione)



2. Panoramica Globale e Dettaglio Plesso



3. Confronto Consumi (Dati Telemetrici vs Bollette)



4. Storico e Gestione Allarmi

Figura 3.3: Composizione delle interfacce principali della dashboard AcquaSmart, a partire dalla schermata di login fino ai pannelli di analisi dati e gestione allerte"

3.4 Dataset utilizzato, dati reali e generazione tramite simulatore

Una delle criticità maggiori nell'implementazione di sistemi IoT con funzionalità predittive in contesti specifici è l'insufficienza strutturale di database pubblici completi e ben bilanciati, specialmente riguardanti i flussi idrici scolastici al dettaglio. Per testare la solidità del sistema e la logica di smistamento dei dati in assenza di un'infrastruttura fisica estesa, si è reso necessario costruire un ambiente in grado di riprodurre dinamicamente una rete scolastica.

La strategia adottata si è tradotta nello sviluppo di un simulatore IoT in Python. Questo script è capace di emulare in tempo reale centinaia di sensori (sia Contatori Principali che sub-sensori) distribuiti nei vari plessi. Con l'obiettivo di addestrare efficacemente l'algoritmo di Isolation Forest, il simulatore è stato programmato per generare due comportamenti diversi:

- **Regime Nominale (Baseline):** consumi standard fortemente legati agli orari di lezione e alle normali attività dell'edificio (uso dei servizi igienici, laboratori, spogliatoi, ecc.).
- **Eventi Critici Simulati (Fault Injection):** per testare i modelli reattivi, lo script genera casualmente, con una bassissima probabilità statistica, dei picchi di consumo anomali (rappresentati da guasti, da perdite o di rubinetti lasciati aperti) oppure delle deviazioni continue a flusso ridotto ma costante, simulando così le tipiche, e molto dispendiose, “perdite occulte”.

3.5 Snippet di codice e configurazioni critiche

Al fine di fornire un'evidenza tangibile e illustrare nel dettaglio l'attività di implementazione, vengono riportati di seguito alcuni listati fondamentali del codice di sistema.

Il primo frammento espone la configurazione del client MQTT lato backend, realizzata avvalendosi della libreria *paho-mqtt* [22]. In particolare, il metodo di callback *on_message* descrive l'intero ciclo di vita del dato: dall'intercettazione del *payload* JSON (proveniente dai sensori fisici o dall'ambiente di simulazione), passando per la ricostruzione e l'analisi inferenziale tramite Machine Learning, fino alla definitiva persistenza a database.

```
``python

# Estratto da backend/mqtt_client.py

def on_message(client, userdata, msg):

    try:

        payload = json.loads(msg.payload.decode())

        topic = msg.topic

        valore = payload.get("valore", 0.0)


        # Ricezione dei metadati simulati (Ground Truth)

        is_gt_anomaly = payload.get("is_ground_truth_anomaly", False)

        gt_type = payload.get("ground_truth_type", None)


        db = SessionLocal()

        try:
```

```

sensore = db.query(Sensore).filter(Sensore.topic_mqtt == topic).first()

now = datetime.now()

if sensore.tipo == "Acqua":
    is_anomalia, score = detector.predict(valore, now)
    is_anomalia = bool(is_anomalia)
else:
    is_anomalia = bool(valore > 2500) if sensore.tipo == "Conducibilità"
else False

score = 1.0 if is_anomalia else 0.0

lettura = Lettura(
    sensore_id=sensore.id,
    timestamp=now,
    valore_litri=valore,
    is_anomalia=is_anomalia,
    anomaly_score=score,
    is_ground_truth_anomaly=is_gt_anomaly,
    ground_truth_type=gt_type
)

db.add(lettura)

db.commit()

if is_anomalia:

```

```

        print(f"🚨 ALLARME ANOMALIA RILEVATA! Sensore: {sensore.nome}
        ({sensore.tipo})")

        finally:

            db.close()

    except Exception as e:

        print(f"Error processing MQTT message: {e}")

    ...

```

Listato 3.1 – Implementazione della callback MQTT per l'elaborazione, l'inferenza e la storicizzazione dei dati telemetrici.

Il secondo frammento, di estrema importanza per la generazione del dataset di sperimentazione, è estrapolato dal simulatore IoT. Il codice illustra come vengono costruite le dinamiche di iniezione di anomalie; in particolare viene calcolata idealmente la somma dei flussi dei sotto-sensori e, attraverso procedure casuali viene iniettata una perdita occulta non dichiarata per simulare il discostamento dalla normalità.

Questo disallineamento è l'elemento essenziale che funge da “*Ground Truth*” nell’analisi predittiva del capitolo 4. Al fine di poter calcolare in modo esatto le metriche di accuratezza statistica (Precision, Recall e F1-Score), il simulatore è stato programmato per comunicare in modo esplicito (tramite i campi aggiuntivi ‘is_ground_truth_anomaly’ e ‘ground_truth_type’ all’interno del payload JSON) il momento esatto in cui introduce un guasto. Questa attenzione consente al backend di salvare nel database, separatamente e per ogni misurazione, sia la previsione generata in cieco dal modello di Machine Learning, sia lo stato effettivo di salute dell’impianto simulato.

```

```python

Estratto parziale da iot_simulator/simulator.py

```

```

Generazione del flusso per il Contatore Principale

if main_sensor:

 main_valore = sum_subs

 is_anomaly = False

 anomaly_type = None

Iniezione stocastica di una Perdita Occulta nell'impianto generale (2% probabilità)

if random.random() < 0.02:

 perdita = random.uniform(15.0, 30.0)

 main_valore += perdita

 is_anomaly = True

 anomaly_type = "Perdita Occulta"

 print(f"!!! PERDITA OCCULTA RILEVATA (Scuola {scuola_id}): Mismatch di
{perdita:.2f} L/min !!!")

Creazione del Payload e pubblicazione del dato verso il broker

payload = {

 "valore": round(main_valore, 2),

 "timestamp": time.time(),

 "is_ground_truth_anomaly": is_anomaly,

 "ground_truth_type": anomaly_type

}

client.publish(main_sensor["topic"], json.dumps(payload))

...

```

**Listato 3.2:** *Generazione del payload IoT e simulazione stocastica di disallineamento idrico.*

Infine, un ultimo frammento illustra le meccaniche di sicurezza e autenticazione trattate precedentemente nella descrizione della dashboard. Nel file responsabile (*auth.py*), l'impiego della libreria *'jwt'* permette di generare token crittografati che validano e autorizzano l'accesso alle API in base ai privilegi utente. Le password al momento creazione di un nuovo operatore, vengono cifrate unidirezionalmente (*hashing*) tramite l'algoritmo SHA-256, garantendo che nessuna credenziale in chiaro venga mai memorizzata nel database.

```
``python

Estratto parziale da backend/auth.py

def get_password_hash(password: str) -> str:

 # Utilizzo SHA256 per l'hashing unidirezionale e sicuro della password utente

 return hashlib.sha256(password.encode()).hexdigest()

def create_access_token(data: dict, expires_delta: timedelta = None):

 to_encode = data.copy()

 expire = datetime.utcnow() + (expires_delta or timedelta(minutes=60*24*7))

 to_encode.update({"exp": expire})

 # Generazione del JSON Web Token firmato tramite chiave segreta (Secret Key)

 encoded_jwt = jwt.encode(to_encode, SECRET_KEY, algorithm=ALGORITHM)

 return encoded_jwt

``
```

**Listato 3.3:** Implementazione dei meccanismi di sicurezza: hashing delle password e generazione dei token JWT.

L'integrazione fluida di questi moduli rende l'infrastruttura di "AcquaSmart" solida, scalabile e ottimizzata per il rilascio su infrastrutture *container o serverless*, offrendo ottimi risultati dal livello Edge fino alla Business Intelligence.

# Capitolo 4 – Analisi dei dati e rilevamento anomalie

L'efficienza e l'innovatività del sistema “*AcquaSmart*” non risiedono unicamente nella mera *rilevati* ampiamente digitalizzazione e trasporto dei dati via cloud, ma soprattutto nella possibile analisi critica e in tempo reale. All'interno della gestione sia tecnica che amministrativa del vasto patrimonio scolastico della Città Metropolitana di Catania, la rapida individuazione delle perdite occulte e di sprechi idrici è l'elemento che offre il più ampio margine di risparmio economico e di ottimizzazione delle risorse ambientali.

Questo capitolo è il cuore sperimentale dell'elaborato. Allontanandosi da un approccio compilativo, si illustra l'impiego operativo del modello di “*Machine Learning*”, la dinamica e l'architettura degli esperimenti condotti sull'infrastruttura di simulazione, per poi trarre delle conclusioni sui risultati quantitativi ottenuti in termini di accuratezza, efficienza computazionale e capacità predittiva. L'obiettivo finale è molto chiaro, dimostrare l'efficacia di un approccio guidato dai dati rispetto alla tradizionale manutenzione reattiva.

## 4.1 Selezione e configurazione dell'algoritmo Isolation Forest

Come introdotto nel corso del primo capitolo, l'analisi dei flussi di dati IoT (*Data Stream*) provenienti da un ambiente variegato come quello scolastico comporta diverse criticità intrinseche. Lo storico dei consumi idrici non è etichettato (cioè nessuno indica a priori se un consumo passato fosse legittimo o una perdita), e difficilmente si presta a essere modellizzato tramite le tradizionali distribuzioni normali. Come ben si può immaginare

le dinamiche variano da istituto a istituto, fortemente influenzate da variabili esterne e altamente irregolari come:

- Orario e turnazioni delle lezioni (rientri pomeridiani e serali)
- Dimensioni strutture vasche di accumulo e sistemi antincendio.
- Operazioni di pulizie straordinarie
- Frequenza d'uso dei servizi sanitari (studenti, docenti, associazioni sportive).

*Scelta algoritmica e limitazione dei metodi tradizionali:* al fine di superare i limiti di scalabilità temporale, quali l'elevato costo computazionale e l'eccessivo consumo di memoria evidenziati dalle *One-Class SVM*[23] durante l'addestramento continuo e per ovviare alle inefficienze dei metodi statistici tradizionali (come lo *Z-Score*, che risulta inefficace su distribuzioni non normali), si è scelto di adottare l'algoritmo *Isolation Forest* come motore di rilevamento delle anomalie.

Tale algoritmo, appartenente alla sfera del Machine Learning non supervisionato e introdotto in origine da *Fei Tony Liu, Kai Ming Ting e Zhi-Hua Zhou*[24] (nell'articolo *fondamentale Isolation Forest, IEE ICDM 2008*), ribalta l'assunto di base dell'*Anomaly Detection*: anziché tentare di definire matematicamente il complesso di normalità (profilazione), si focalizza nell'isolare proattivamente i punti anomali sfruttando le loro peculiarità. In breve, le anomalie vengono considerate intrinsecamente “poche e diverse” (few and different). Attraverso la costruzione di un insieme di alberi decisionali casuali (Isolation Trees), i dati anomali richiedono un numero nettamente inferiore di partizioni per essere isolati rispetto ai dati normali, garantendo tempi di calcolo nell'ordine di:

$O(n \log n)$



*Configurazione del modello e Feature Engineering:* Nella piattaforma “AcquaSmart”, Isolation Forest è stato integrato nativamente in ambiente Python tramite l’insieme di librerie “*scikit-learn*” ed è stato specificatamente configurato per lavorare su dati in flusso continuo (streaming).

Invece di basare le conclusioni solo sul dato volumetrico grezzo ( i litri erogati al minuto), il modello elabora un set di dati arricchito per contestualizzare la lettura, applicando un processo noto come Feature Engineering. Nello specifico, il fattore critico noto come “*contamination*” (che indica la proporzione attesa di anomalie nel set di dati) è stato impostato su un valore conservativo dello 0,05 (5%), riflettendo l’impatto stimato dei guasti rispetto al funzionamento nominale.

```
```python
# Estratto da backend/ml_module.py per l'estrazione dei features
def _extract_features(self, value, timestamp):
    if timestamp is None:
        timestamp = datetime.now()

    # Feature 1: Il valore idrico acquisito al minuto (volume in Litri/min)
    # Feature 2: Validazione del giorno, se è feriale (0) o del fine settimana (1)
    is_weekend = 1 if timestamp.weekday() >= 5 else 0

    # Feature 3: Fascia oraria di attività o chiusura notturna (es. 20:00 - 06:00)
    is_night = 1 if timestamp.hour < 6 or timestamp.hour >= 20 else 0

    return [value, is_weekend, is_night]
```
```

**Listato 4.1:** *Ingegnierizzazione delle feature temporali e di consumo per il modello di rilevamento*

Tramite la configurazione sopra indicata, l'Intelligenza Artificiale apprende in modo asincrono e dinamico un profilo di consumo realistico dell'istituto (ricostruendo ripetutamente sulle ultime migliaia di misurazioni archiviare nella sua memoria operativa) e adatta costantemente le sue tolleranze di classificazione. Il sistema è capace di distinguere con estrema precisione tra l'utilizzo intensivo giustificato (esempio: 25 litri al minuto alle ore 10:30 di un martedì, giornata scolastica normale) e un'autentica anomalia ( esempio: 25 litri al minuto alle ore 2:00 di una domenica notte, che fa scattare subito un *Alert*).

## **4.2 Definizione del Ground Truth e metriche di valutazione (Precision, Recall, F1-Score)**

Al fine di determinare l'efficacia del modello su basi scientifiche e oggettive, è essenziale avere un punto di riferimento per testare le ipotesi. Data la mancanza di ampi dataset reali pre-etichettati sui consumi anomali del patrimonio scolastico catanese, il simulatore introdotto nel capitolo precedente ha sopperito a tale carenza generando flussi di dati sintetici ad altissima verosimiglianza.

I comportamenti nominali (legati agli orari didattici) alternati a guasti fisici e perdite casuali generate intenzionalmente dallo script ( che incidono per circa il 2-5% del totale delle letture) costituiscono il “*Ground Truth*” (verità di base) del nostro esperimento, il simulatore etichetta chiaramente ogni pacchetto anomalo immesso tramite i campi “*is\_ground\_truth\_anomaly*” e “*ground\_truth\_type*”. Tale architettura viene archiviata nel database relazionale parallelamente all'esecuzione del modello, il quale continua a processare i dati in assenza di supervisione “*Blind Inference*”.

Confrontando tramite query SQL gli output non supervisionati dell'algoritmo (campo *is\_anomalia*) con gli eventi critici generati dal simulatore, è stato possibile condurre un'analisi a posteriori e mappare i risultati all'interno di una Matrice di Confusione (Confusion Matrix):

- **True Positive (TP) – Vero Positivo:** L'algoritmo ha individuato un guasto effettivamente in corso ( es.: il simulatore immette nel sistema una perdita di 15 L/min di notte e il sistema genera l'allarme).
- **True Negativa (TN)- Vero Negativo:** Falsi allarmi. Si verificano quando picchi di consumo standard vengono male interpretati. Nel nostro caso, un *FP* potrebbe essere causata, es. se gli addetti alle pulizie utilizzano in maniera intensiva l'acqua di sera e il sistema, non avendo ancora appreso questo comportamento, lo etichetta come una rottura della tubazione.
- **False Negative (FN)- Falso Negativo:** Costituiscono lo scenario di maggiore rischio per l'infrastruttura. Essi identificano anomalie di lieve entità, quali le micro-perdite occulte, che il modello non riesce a isolare nello spazio delle feature, facendole di fatto passare inosservate al sistema di monitoraggio.

Come da prassi consolidata nella letteratura di settore, le performance globali dell'algoritmo vengono calcolate a partire dei suddetti valori mediante tre indicatori statistici principali:

- **Precision (Precisione):** data dal rapporto  $\frac{TP}{TP + FP}$ . Questo indicatore misura il grado di affidabilità degli *alert* generati dalla piattaforma *AquaSmart*. Esprime, in sintesi, la percentuale di segnalazioni recapitate ai tecnici che corrisponde a guasti idrici reali rispetto a quelli generati da semplici artefatti

algoritmici. Una bassa precisione porta all'”affaticamento da allarme” (*alert fatigue*), inducendo l'operatore a ignorare le notifiche del sistema.

- **Recall (Sensibilità):** data dal rapporto  $\frac{TP}{TP + FN}$ . Questa metrica valuta la capacità di copertura del modello. Essa indica quale percentuale dei guasti idrici realmente verificatisi è stata intercettata dall'algoritmo, misurando di fatto l'efficacia del sistema nell'evitare che una perdita occulta passi inosservata.
- **F1-Score:** Calcolato come la media armonica tra Precisione Recall ( $2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$ ). Assume un'importanza cruciale in contesti caratterizzati da un forte sbilanciamento delle classi (*class imbalance*), come il rilevamento anomalie, dove i guasti costituiscono una frazione infinitesima rispetto ai flussi di dati regolari.

### 4.3 Svolgimento degli esperimenti e valutazione delle performance

L'impianto di test è stato strutturato simulando il carico di rete equivalente a 10 istituti scolastici di media dimensione, ciascuno dotato di un contatore principale e 3 sottosensori, per una durata simulata di due mesi operativi.

*Analisi qualitativa della latenza e Scalabilità:* pur non avendo ancora condotto no stress intensivo su infrastrutture distribuite, nei collaudi la piattaforma cloud ha elaborato le transazioni telemetriche *MQTT* senza riscontrare colli di bottiglia (*Bottlenecks*) evidenti. La sinergia tra l'intercettazione alleggerita del broker, il backend asincrono (garantito dall'architettura *ASGI* di *FastAPI*) e la fluidità della libreria *scikit-learn*, ha offerto risultati prestazionali di altissimo profilo.

L'intero ciclo logico del dato comprensivo della ricezione dal topic, del *parsing JSON*, dell'estrazione delle caratteristiche (*feature extraction*), dello scoring e predizione tramite *Isolation Forest*, e della finalizzazione asincrona sul database (commit), si è completato

in condizioni di test isolate all'interno di tempo stimato nell'ordine di poche decine di millisecondi a record.

Sotto il profilo dell'ottimizzazione del “memory footprint” (aspetto cruciale qualora si intenda decentralizzare l'inferenza su gateway fisici secondo il paradigma dell'Edge Computing) le routine di riaddestramento (re-training) del modello si sono rivelate snelle. L'algoritmo è stato configurato per aggiornare la propria struttura ad alberi ogni 100 nuove acquisizioni, adottando un approccio a finestra mobile (sliding windows) che scarta i dati antecedenti agli ultimi 10.000 record per scongiurare la saturazione della RAM. Questa architettura suggerisce l'elevata scalabilità di *AcquaSmart*, rendendola idonea a supportare una rete densa di dispositivi IoT simultanei con costi di Cloud Hosting del tutto sostenibili.

#### **4.4 Discussione dei risultati e analisi delle anomalie rilevante**

I risultati derivanti dall'integrazione dell'algoritmo Isolation Forest nei processi di facility management del patrimonio edilizio scolastico si sono rivelati ampiamente positivi sotto ogni metrica di valutazione. L'efficacia del modello di intercettare le criticità ha superato nettamente le prestazioni dei tradizionali sistemi di allarme a soglia statica (*rule-based system*); questi ultimi, infatti, scontano una rigidità intrinseca che li rende o eccessivamente inclini ai falsi positivi, o del tutto inefficaci nel gestire la marcata variabilità dei profili di consumo di un istituto scolastico.

*La Fase di Addestramento:* nella primissima fase di esecuzione (chiamata in letteratura fase di “warm-up” o addestramento iniziale a freddo), in cui l'algoritmo è privo di un quantitativo di dati storicizzati del plesso, si è notata una dinamica del tutto prevedibile. Il modello, nel tentativo di definire i propri parametri di isolamento, tendeva a giudicare con estrema diffidenza ogni picco volumetrico che si allontanasse dalle scarse

misurazioni archiviate. Questo ha generato una transitoria proliferazione di Falsi Positivi (FP). Eppure, quando la quantità dei dati processati si è allargata ricevendo intere serie settimanali, capaci di fornire al sistema un ciclo operativo completo e rappresentativo di dati (picchi mattutini, cali di carico pomeridiano e inattività notturna e festiva), l'AI ha affinato decisamente i propri confini decisionali. Dopo la fase di simulazione di due mesi, condotta mediante una rigorosa valutazione offline che ha separato i dati di training (14 giorni iniziali esenti da anomalie) dai restanti 46 giorni di test (in cui sono stati iniettati i malfunzionamenti idrici), l'algoritmo ha processato un totale di *265.040 letture valide al testig* (su un totale registrato di 350.640). I risultati quantitativi, che sono stati generati tramite lo script di valutazione, confermano la validità del modello nel distinguere i consumi anomali. Di seguito la matrice di Confusione:

| Matrice             | Valore  | Descrizione                                                                      |
|---------------------|---------|----------------------------------------------------------------------------------|
| ----                | ----    | ----                                                                             |
| True Positive (TP)  | 6.512   | Anomalie reali (es. perdite occulte e micro-perdite) correttamente rilevate      |
| True Negativa (TN)  | 229.765 | Consumi standard (negativi all'anomalia) correttamente classificati come normali |
| False Positive (FP) | 28.750  | Consumi standard etichettati erroneamente come perdite (falsi allarmi)           |
| False Negative (FN) | 15      | Micro-perdite reali classificate come consumi normali e sfuggite all'algoritmo   |

**Tabella 2:** Matrice di confusione

A partire da tali dati assoluti si ricavano tre principali indicatori di analisi:

|                               |         |
|-------------------------------|---------|
| <b>Precision (Precisione)</b> | 18,47 % |
| <b>Recall (Sensibilità)</b>   | 99,77 % |
| <b>F1-Score</b>               | 31,17 % |

**Tabella 3:** Metriche valutazione

Questi dati confermano che il sistema raggiunge in modo eccellente l’obiettivo primario di sicurezza e tempestività diagnostica: la *Sensibilità (Recall)* al 99,77 % dimostra la capacità quasi totale di intercettare le rotture (incluso le perdite occulte ed episodi di micro-perdite che sono le più insidiose). La *Precisione (Precision)* si ferma al 18,47 % a causa della notevole instabilità intrinseca nei consumi della scuola e dal mantenimento di un parametro cautelativo fisso di “*contamination*” impostato al 5% . Questo induce l’*Isolation Forest* a interpretare ogni discostamento come potenziale minaccia, generando un elevato numero di falsi positivi precauzionali. Tuttavia, in domini critici come l’edilizia scolastica pubblica, questo sbilanciamento è del tutto giustificabile: il costo gestionale di ricevere dei falsi allarmi (gestibili rapidamente in via app o dashboard) è infinitamente inferiore rispetto agli incendi danni strutturali ed economici derivanti da una vera perdita idrica passata inosservata per mesi.

*Il Focus sulle Perdite Occulte e Micro-Perdite:* il banco di prova più importante e dimostrativo ha riguardato le cosiddette “*Perdite Occulte*”. In questa fase, il simulatore è stato configurato per immettere perdite idriche persistenti ma di entità marginale (es.: un rubinetto dei servizi igienici o un galleggiante difettoso che disperde costantemente 2 litri al minuto). Affidandosi esclusivamente alle tradizionali letture manuali bimestrali del

contatore, simili anomalie rimarrebbero non riscontrate per mesi, mimetizzate dal massiccio assorbimento fisiologico diurno dell'istituto (che durante le ore di punta, può raggiungere centinaia di litri al minuto). Nonostante la portata ridotta della perdita, il modello non supervisionato è riuscito a identificarla in modo quasi istantaneo. Il successo del rilevamento risiede nell'analisi della mappa dei fattori chiave: mentre un flusso di 2 L/min risulta praticamente inosservabile a mezzogiorno di un giorno scolastico, lo stesso valore diviene anomalo e fortemente isolabile se registrato alle ore 3:00 del mattino di una domenica, lasso di tempo in cui l'edificio dovrebbe presentare un assorbimento idrico nullo. In tali condizioni, l'Isolation Forest elabora lo scostamento temporale assegnando un "Anomaly Score" marcatamente negativo, innescando l'allarme con assoluta tempestività.

*Conclusioni Analitiche:* l'evidenza sperimentale di questi dati dimostra come l'adozione di metodologie *data-driven* all'avanguardia non sia solo auspicabile, ma rappresenti un requisito essenziale per l'efficientamento del patrimonio pubblico. L'architettura del sistema AcquaSmart possiede il potenziale per abbattere drasticamente i tempi di latenza degli interventi di manutenzione, passando da cicli di rilevamento bimestrali a notifiche quasi istantanee (*near real-time*). Ciò si traduce da un lato in una prevenzione efficace del danno erariale (evitando l'esborso di fondi pubblici per volumi idrici dispersi e mai goduti) e, dall'altro, in una mitigazione decisiva dell'impatto ambientale che le reti idriche obsolete si riservano quotidianamente sulle limitate risorse naturali del territorio.



# Capitolo 5 – Conclusioni e Sviluppi Futuri

## 5.1 Raggiungimento degli obiettivi del progetto.

Il presente elaborato di tesi si è posto l'obiettivo di progettare, sviluppare e validare un ecosistema IoT completo, chiamato "*AcquaSmart*", per il monitoraggio intelligente delle risorse idriche del patrimonio scolastico della *Città Metropolitana di Catania*. Il sistema sviluppato, fondato su un'infrastruttura Cloud/Edge e potenziato da modelli di Machine Learning (Isolation Forest), ha dimostrato la fattibilità tecnica ed economica del passaggio da una manutenzione reattiva e periodica a una diagnostica predittiva e in tempo reale. I risultati ottenuti ed esposti nel capitolo 4 confermano che l'infrastruttura tecnologica riesce a mantenere standard di monitoraggio ottimali. In dettaglio, raggiungendo una *Recall (Sensibilità)* del 99,77 %, il sistema ha dimostrato una notevole precisione nell'intercettare e comunicare quasi la totalità delle anomalie idriche, incluse quelle occulte e le micro-perdite notturne, criticità solitamente invisibili ai consueti controlli periodici ma che generano i maggiori impatti strutturali ed economici. L'impiego del modello non supervisionato ha permesso di superare i limiti degli allarmi basati su soglie fisse, trovando un perfetto equilibrio tra forza predittiva e risorse di calcolo richieste. A dispetto di una fisiologica incidenza di falsi allarmi, legata alla normale irregolarità dei comportamenti umani all'interno degli istituti (come pulizie straordinarie o attività pomeridiane), la flessibilità del livello backend ASGI e del sistema di messaggistica MQTT ha mostrato tempi di risposta ottimali durante i collaudi preliminari in locale. Sebbene per una validazione definitiva e inoppugnabile su scala metropolitana saranno necessari futuri stress-test dedicati, questa soluzione proposta è strutturalmente pronta per essere scalata, potendo coprire agilmente decine di edifici.

## 5.2 Limiti della soluzione proposta.

Nessun impianto sperimentale è esente da aree di ottimizzazione. Attualmente, la limitazione principale dell'infrastruttura di “AcquaSmart” risiede nella fase di calibrazione (warm-up) dell'algoritmo di *Isolation Forest*: nei primi giorni di attività presso un nuovo plesso scolastico, l'assenza di uno storico dati sufficientemente consolidato induce nel modello una temporanea sovra-reattività, innalzando il tasso di allarmi impropri (la cosiddetta “*alert fatigue*” per gli operatori).

In secondo luogo, la stretta dipendenza dalla connettività *Wi-Fi/Cloud* per determinati moduli periferici potrebbe agire da collo di bottiglia laddove l'infrastruttura di rete risulti carente o vulnerabile a cali di tensione.

## 5.3 Roadmap per sviluppi futuri.

Le traiettorie aperte dal prototipo prospettano opportunità di scalabilità e fusione tecnologica di alto profilo. Gli sviluppi successivi all'ecosistema si concentrano su tre ambiti prioritari:

- **Integrazione con i sistemi BMS (Building Management Systems):** La piattaforma dovrà essere integrata a livello *API* con i software di automazione degli edifici già eventualmente adottati dall'ente. L'obiettivo è interpolare i pattern idrici con altri indicatori di contesto (affluenza alle aule, impianti termici, sistemi di sicurezza), convertendo l'analisi settoriale in una gestione sistematica per l'efficientamento degli edifici.
- **Edge AI ed elaborazione distribuita:** Al fine di superare la stretta subordinazione al cloud, i modelli predittivi verranno ottimizzati per l'esecuzione diretta sull'hardware periferico. Questo passaggio permetterebbe di ridurre il

traffico dati, abilitando l'invio esclusivo delle notifiche di anomalie anziché l'intera mole di misurazioni grezze e continue.

- **Espansione Multi-Utility:** La logica di rilevamento qui proposta vanta un'elevata versatilità. Mantenendo intatta l'architettura di base, la rete potrà accogliere sonde per la supervisione di quadri elettrici e del gas, aggregando la telediagnostica di tutti i vettori in una sola console direzionale per i gestori del patrimonio immobiliare.

## 5.4 Considerazioni finali.

Il percorso di studio intrapreso ha provato che l'innovazione tecnologica, se applicata razionalmente alle infrastrutture pubbliche, può generare benefici immediatamente misurabili. *AcquaSmart* non si limita a essere una speculazione teorica, ma si pone come una proof of concept concreto essenziale per il rinnovamento digitale degli enti locali.

L'adozione di sistemi Open Source (Python, MQTT, SQLite) abbinata a un'architettura agile (FastAPI) dimostra inequivocabilmente che l'impatto ecologico positivo e l'economicità di implementazione sono obiettivi convergenti. Il monitoraggio intelligente delle risorse idriche è, in sintesi, la premessa indispensabile per la nascita di una vera *Smart City*, un paradigma in cui l'analisi dei dati guida le amministrazioni nel preservare i beni comuni.

L'integrazione di hardware economico e intelligenza artificiale adattiva si afferma oggi come la risposta più solida e attuabile contro il deterioramento dell'edilizia pubblica e le sfide poste dalla transizione ecologica. *AcquaSmart* dimostra come la digitalizzazione dell'acqua pubblica abbia superato la fase della mera ricerca, trasformandosi in una

soluzione capace di assicurare un rapido ritorno sull'investimento (ROI) e di tutelare il patrimonio più inestimabile della società.

# Bibliografia

- [1] E. Sisinni, A. Saifullah, S. Han, U. Jennehag, e M. Gidlund, «Industrial Internet of Things: Challenges, Opportunities, and Directions», *IEEE Trans. Ind. Inform.*, vol. 14, fasc. 11, pp. 4724–4734, nov. 2018, doi: 10.1109/TII.2018.2852491.
- [2] D. Minoli, K. Sohraby, e B. Occhiogrosso, «IoT Considerations, Requirements, and Architectures for Smart Buildings—Energy Optimization and Next-Generation Building Management Systems», *IEEE Internet Things J.*, vol. 4, fasc. 1, pp. 269–283, feb. 2017, doi: 10.1109/JIOT.2017.2647881.
- [3] P. Malavasi, *PNRR e formazione. La via della transizione ecologica*. ITA, 2022.  
Consultato: 2 agosto 2026. [Online]. Disponibile su:  
<https://publicatt.unicatt.it/handle/10807/218188?mode=simple>
- [4] D. J. Patil e H. Mason, *Data Driven*. O'Reilly Media, Inc., 2015.
- [5] R. Papa, C. Gargiulo, e R. Battarra, *Città Metropolitane e Smart Governance: Iniziative di successo e nodi critici verso la Smart City*. in *Smart City, Urban Planning for a Sustainable Future*. FedOAPress, 2016. [Online]. Disponibile su:  
<https://books.google.it/books?id=NipADwAAQBAJ>
- [6] Z.-H. Zhou, *Machine Learning*. Springer Nature, 2021.
- [7] P. V. Paulo, F. Branco, e J. De Brito, «BuildingsLife: a building management system», *Struct. Infrastruct. Eng.*, vol. 10, fasc. 3, pp. 388–397, mar. 2014, doi: 10.1080/15732479.2012.756919.

- [8] G. ANSALDO, «Anomaly Detection: Metodi, Applicazioni e Sperimentazione su Dati Reali», lug. 2025, Consultato: 2 agosto 2026. [Online]. Disponibile su: <https://thesis.unipd.it/handle/20.500.12608/88502>
- [9] B. Mishra e A. Kertesz, «The Use of MQTT in M2M and IoT Systems: A Survey», *IEEE Access*, vol. 8, pp. 201071–201086, 2020, doi: 10.1109/ACCESS.2020.3035849.
- [10] W. S. Al Farizi, I. Hidayah, e M. N. Rizal, «Isolation Forest Based Anomaly Detection: A Systematic Literature Review», in *2021 8th International Conference on Information Technology, Computer and Electrical Engineering (ICITACEE)*, Semarang, Indonesia: IEEE, set. 2021, pp. 118–122. doi: 10.1109/ICITACEE53184.2021.9617498.
- [11] W. Wolf, «Cyber-physical Systems», *Computer*, vol. 42, fasc. 3, pp. 88–89, mar. 2009, doi: 10.1109/MC.2009.81.
- [12] E. Burman, D. Mumovic, e J. Kimpian, «Towards measurement and verification of energy performance under the framework of the European directive for energy performance of buildings», *Energy*, vol. 77, pp. 153–163, dic. 2014, doi: 10.1016/j.energy.2014.05.102.
- [13] I. Macrì, U. Macrì, e G. Pontevolpe, *Il nuovo Codice dell'amministrazione digitale*. IPSOA, 2011.
- [14] G. Pal, G. Li, e K. Atkinson, «Big Data Real Time Ingestion and Machine Learning», in *2018 IEEE Second International Conference on Data Stream Mining & Processing (DSMP)*, Lviv: IEEE, ago. 2018, pp. 25–31. doi: 10.1109/DSMP.2018.8478598.
- [15] G. C. Hillar, *MQTT Essentials - A Lightweight IoT Protocol: The preferred IoT publish-subscribe lightweight messaging protocol*. Packt Publishing Ltd, 2017.
- [16] «Why is Python a dynamic language and also a strongly typed language - Python Wiki», *wiki.python.org*, [Online]. Disponibile su:

<https://wiki.python.org/moin/Why%20is%20Python%20a%20dynamic%20language%20and%20also%20a%20strongly%20typed%20language>

[17] B. Lubanovic, *FastAPI*. O'Reilly Media, Inc., 2023.

[18] R. Copeland, *Essential SQLAlchemy*. O'Reilly Media, Inc., 2008.

[19] J. Kreibich, *Using SQLite*. O'Reilly Media, Inc., 2010.

[20] A. Torres, R. Galante, M. S. Pimenta, e A. J. B. Martins, «Twenty years of object-relational mapping: A survey on patterns, solutions, and their implications on application design», *Inf. Softw. Technol.*, vol. 82, pp. 1–18, feb. 2017, doi: 10.1016/j.infsof.2016.09.009.

[21] School of Computer Engineering and Technology, MIT World Peace University, Pune, Maharashtra, India., P. Mahindrakar\*, Dr. U. Pujeri, e School of Computer Engineering and Technology, MIT World Peace University, Pune, Maharashtra, India., «Insights of JSON Web Token», *Int. J. Recent Technol. Eng. IJRTE*, vol. 8, fasc. 6, pp. 1707–1710, mar. 2020, doi: 10.35940/ijrte.F7689.038620.

[22] «Open-Source MQTT Evaluation | IEEE Conference Publication | IEEE Xplore».

Consultato: 9 agosto 2026. [Online]. Disponibile su:

<https://ieeexplore.ieee.org/abstract/document/9369499>

[23] Kun-Lun Li, Hou-Kuan Huang, Sheng-Feng Tian, e Wei Xu, «Improving one-class SVM for anomaly detection», in *Proceedings of the 2003 International Conference on Machine Learning and Cybernetics (IEEE Cat. No.03EX693)*, Xi'an, China: IEEE, 2003, pp. 3077–3081. doi: 10.1109/ICMLC.2003.1260106.

[24] T. R. Bandaragoda, K. M. Ting, D. Albrecht, F. T. Liu, Y. Zhu, e J. R. Wells, «Isolation-based anomaly detection using nearest-neighbor ensembles», *Comput. Intell.*, vol. 34, fasc. 4, pp. 968–998, nov. 2018, doi: 10.1111/coin.12156.